PRAXISHANDBUCH

John the Ripper

Das Praxishandbuch

Passwort-Hashes verstehen und offline prüfen – von der Wortliste bis zur eigenen Regel

```
$ john --wordlist=testliste.txt --rules testhashes.txt
Loaded 3 password hashes with 3 different salts
testpass123      (eugen)
1g 0:00:00:00 DONE
$ john --show testhashes.txt
eugen:testpass123
```

VERSTEHEN, WARUM EIN PASSWORT HÄLT. ODER EBEN NICHT.

Mit Übungen, Spickzettel, Glossar und Index · Ausgabe Oktober 2026

John the Ripper
Das Praxishandbuch

Passwort-Hashes verstehen und offline prüfen – von der Wortliste bis zur eigenen Regel

Von Hashing und Salting über Angriffsmodi und Regeln bis zu Performance und Session-Verwaltung.
Mit geführten Übungen mit selbst erzeugten Testhashes, Spickzettel, Glossar und Index.

Inhaltsverzeichnis

Vorwort 5

Für wen ist dieses Handbuch? 5

Aufbau 6

Kapitel 1: Was ist John the Ripper? 7

1.1 Einordnung: offline statt online 7

1.2 Typische Einsatzgebiete 7

1.3 Der grundsätzliche Ablauf 7

1.4 Dual-Use wie jedes Werkzeug dieser Reihe 8

Zusammenfassung 8

Kapitel 2: Grundlagen – Hashing, Salting und warum Cracking offline stattfindet 9

2.1 Warum Hashes statt Klartext? 9

2.2 Salting als Gegenmaßnahme 9

2.3 Warum „Cracken" eigentlich Raten und Vergleichen ist 10

Zusammenfassung 10

Kapitel 3: Installation und Laborumgebung 11

3.1 Installation 11

3.2 Eigene Testhashes erzeugen 11

3.3 Eine Testwortliste bereitstellen 12

3.4 Laborumgebung 12

Zusammenfassung 12

Kapitel 4: Angriffsmodi 13

4.1 Die drei Grundmodi im Überblick 13

4.2 Single Crack 13

4.3 Wortlisten-Modus 13

4.4 Incremental-Modus (kurzer Vorgriff) 14

Zusammenfassung 14

Kapitel 5: Hash-Formate erkennen und extrahieren 15

5.1 Hash-Formate an der Struktur erkennen 15

5.2 Hashes aus Linux-Systemdateien extrahieren 15

5.3 Hashes aus Dateien extrahieren: die *2john-Werkzeuge 16

Zusammenfassung 16

Kapitel 6: Regeln – Wortlisten intelligent erweitern . . . 17

6.1 Warum Regeln? . . . 17

6.2 Vordefinierte Regelsätze . . . 17

6.3 Eine eigene, einfache Regel . . . 18

Zusammenfassung . . . 18

Kapitel 7: Masken und der Incremental-Modus im Detail . . . 19

7.1 Incremental-Modus: der vollständige, aber teure Weg . . . 19

7.2 Masken: gezielt statt vollständig . . . 19

7.3 Masken aus bereits geknackten Passwörtern ableiten . . . 20

Zusammenfassung . . . 20

Kapitel 8: Performance, Sessions und die Potfile-Verwaltung . . . 21

8.1 Was die Geschwindigkeit bestimmt . . . 21

8.2 Sessions: lange Läufe unterbrechen und fortsetzen . . . 21

8.3 Die Potfile . . . 22

Zusammenfassung . . . 22

Kapitel 9: John the Ripper und Hashcat im Vergleich . . . 23

9.1 Zwei verwandte, aber unterschiedlich ausgerichtete Werkzeuge . . . 23

9.2 Dieselben Konzepte, andere Syntax . . . 23

9.3 Wann welches Werkzeug? . . . 24

Zusammenfassung . . . 24

Kapitel 10: Praxisteil – Laboraufbau und geführte Übungen . . . 25

10.1 Laboraufbau . . . 25

10.2 Checkliste: sicherer Umgang mit Testdaten . . . 25

10.3 Übungen . . . 25

Zusammenfassung . . . 27

Kapitel 11: Troubleshooting, Spickzettel, Glossar und Ressourcen . . . 28

11.1 Häufige Probleme . . . 28

11.2 Spickzettel . . . 29

11.3 Glossar . . . 29

11.4 Weiterführende Ressourcen . . . 30

Zusammenfassung . . . 30

Verzeichnis der Befehlsbeispiele . . . 31

Index **32**

Vorwort

`John the Ripper` (kurz oft „John" oder „JtR") ist eines der ältesten und am weitesten verbreiteten Werkzeuge zum **Prüfen von Passwort-Hashes** gegen Wortlisten, Regeln und systematische Kombinationen. Wo Wireshark zeigt, was tatsächlich über die Leitung geht, und hping3/Nmap zeigen, wie sich Netzwerke erkunden lassen, beantwortet John the Ripper eine andere, ebenso zentrale Frage der IT-Sicherheit: Wie widerstandsfähig ist ein Passwort wirklich, wenn sein Hash in die falschen Hände gerät?

Dieses Handbuch ergänzt die anderen Bände dieser Reihe um die Offline-Analyseseite der Authentifizierung. Es richtet sich an dieselbe Zielgruppe: IT-Administrator:innen, Studierende, angehende Penetration Tester und alle, die verstehen wollen, warum manche Passwort-Richtlinien wirken und andere nicht.

> **ILLEGAL – niemals ohne Auftrag**
>
> Wie in den anderen Büchern dieser Reihe gilt ohne Ausnahme: John the Ripper darf **ausschließlich** gegen Passwort-Hashes eingesetzt werden, zu deren Prüfung du ausdrücklich berechtigt bist – aus deinen eigenen Testsystemen, aus einem schriftlich autorisierten Auftrag, oder aus frei verfügbaren Übungsdateien, die ausdrücklich zu Lernzwecken bereitgestellt werden. Das unbefugte Beschaffen oder Knacken fremder Zugangsdaten ist nach § 202a StGB (Ausspähen von Daten) strafbar, unabhängig davon, ob der Versuch erfolgreich ist.

Für wen ist dieses Handbuch?

Du bist hier richtig, wenn du

- bereits das Ethical-Hacking-Handbuch kennst und die dort nur kurz angerissene Passwort-Analyse vertiefen möchtest,
- die eigene Passwort-Richtlinie realistisch einschätzen willst, statt dich auf Bauchgefühl zu verlassen,
- verstehen möchtest, warum Salting und moderne Hash-Verfahren einen so großen Unterschied machen,
- dich auf Zertifizierungen vorbereitest, in denen Passwort-Audits eine Rolle spielen (eJPT, OSCP, CompTIA Security+/PenTest+).

Aufbau

Kapitel	Thema
1	Was ist John the Ripper? Einordnung, Geschichte, Abgrenzung zu Hashcat
2	Grundlagen: Hashing, Salting, warum Cracking offline stattfindet
3	Installation und Laborumgebung
4	Angriffsmodi: Single Crack, Wortliste, Incremental
5	Hash-Formate erkennen und extrahieren
6	Regeln: Wortlisten intelligent erweitern
7	Masken und der Incremental-Modus im Detail
8	Performance, Sessions und die Potfile-Verwaltung
9	John the Ripper und Hashcat im Vergleich
10	Praxisteil: Laboraufbau und geführte Übungen
11	Troubleshooting, Spickzettel, Glossar und Ressourcen

> **TIPP – Profi-Trick**
>
> Erzeuge dir für die Übungen in diesem Buch **ausschließlich eigene Testhashes** aus frei erfundenen Testpasswörtern (Kapitel 3 und 10) – du brauchst für kein einziges Beispiel echte, reale Zugangsdaten.

Kapitel 1: Was ist John the Ripper?

Lernziele

- Du kannst John the Ripper von Online-Brute-Force-Werkzeugen wie Hydra abgrenzen.
- Du kennst die Geschichte und die wichtigsten Einsatzgebiete von John the Ripper.
- Du verstehst den grundsätzlichen Ablauf einer Passwort-Prüfung mit Hashes.

1.1 Einordnung: offline statt online

Das Ethical-Hacking-Handbuch dieser Reihe unterscheidet in Kapitel 7.1 zwei Arten von Passwortangriffen: **online** (gegen ein laufendes Login-Formular, z. B. mit Hydra) und **offline** (gegen bereits beschaffte Hashes). John the Ripper ist das klassische Werkzeug für den zweiten Fall.

Angriffsart	Beispielwerkzeug	Charakteristik
Online	Hydra	gegen ein laufendes Login, begrenzt durch Netzwerk und Sperrmechanismen
Offline	John the Ripper, Hashcat	gegen bereits vorliegende Hashes, begrenzt nur durch Rechenleistung

Gut zu wissen

John the Ripper wurde 1996 von Solar Designer (Alexander Peslyak) veröffentlicht, ursprünglich für Unix-Passwort-Hashes. Die seither von der Community gepflegte **Jumbo-Version** (`john-jumbo`) erweitert das Originalprogramm um hunderte zusätzliche Hash-Formate und Werkzeuge und ist heute die in den meisten Distributionen und in Kali vorinstallierte Variante.

1.2 Typische Einsatzgebiete

- **Passwort-Audits**: Wie viele Konten in einer exportierten Hash-Liste lassen sich mit gängigen Wortlisten innerhalb vertretbarer Zeit knacken?
- **Verifikation von Hash-Verfahren**: Zeigen, wie viel schneller schwache (z. B. unsalted MD5) gegenüber starken Verfahren (z. B. bcrypt) tatsächlich zu knacken sind.
- **Post-Exploitation in autorisierten Pentests**: Prüfung erbeuteter Hash-Dumps auf wiederverwendete oder schwache Passwörter (Ethical-Hacking-Handbuch, Kapitel 7).
- **Forensik und Incident Response**: Wiederherstellung eines vergessenen, selbst gesetzten Passworts auf einem eigenen System.

1.3 Der grundsätzliche Ablauf

Beispiel 1.1: Typischer Ablauf einer Passwort-Prüfung (Konzept)

```
1. Hashes beschaffen (nur aus autorisierter Quelle, Kapitel 5)
2. Hash-Format erkennen bzw. angeben (Kapitel 5)
3. Angriffsmodus wählen: Single Crack, Wortliste, Regeln, Incremental (Kapitel 4, 6-7)
4. John the Ripper laufen lassen, Fortschritt und Treffer beobachten (Kapitel 8)
5. Gefundene Passwörter einsehen, Ergebnis auswerten und dokumentieren
```

1.4 Dual-Use wie jedes Werkzeug dieser Reihe

LEGAL – nur mit Auftrag/Einwilligung

John the Ripper hat keine Funktion, die „nur für legale Zwecke" reserviert ist. Der einzige Unterschied zwischen einem legitimen Passwort-Audit und einem Angriff ist – wie im gesamten Ethical-Hacking-Handbuch – die **Berechtigung**: eigene Testsysteme, ein schriftlicher Auftrag mit definiertem Scope, oder ausdrücklich zu Übungszwecken bereitgestellte Hash-Dateien (Kapitel 3 und 10).

Zusammenfassung

Zusammenfassung

- John the Ripper prüft offline beschaffte Passwort-Hashes, während Werkzeuge wie Hydra online gegen ein laufendes Login arbeiten.
- Die von der Community gepflegte Jumbo-Version erweitert das Originalprogramm um zahlreiche Hash-Formate.
- Haupteinsatzgebiete sind Passwort-Audits, Verifikation von Hash-Verfahren, Post-Exploitation und Forensik.
- Wie jedes Werkzeug dieser Reihe entscheidet ausschließlich die Berechtigung über Legalität, nicht die Technik.

Kapitel 2: Grundlagen – Hashing, Salting und warum Cracking offline stattfindet

Lernziele

- Du verstehst, warum Passwörter als Hash statt im Klartext gespeichert werden.
- Du kennst den Zweck von Salting und seine Wirkung auf Cracking-Versuche.
- Du kannst erklären, warum „Cracken" eigentlich systematisches Ausprobieren und Vergleichen ist.

2.1 Warum Hashes statt Klartext?

Eine **Hashfunktion** wandelt eine Eingabe beliebiger Länge in einen Wert fester Länge um, aus dem sich die Eingabe nicht direkt zurückrechnen lässt. Systeme speichern deshalb nicht das Passwort selbst, sondern seinen Hash: Bei der Anmeldung wird das eingegebene Passwort erneut gehasht und mit dem gespeicherten Wert verglichen.

Beispiel 2.1: Derselbe Klartext ergibt immer denselben Hash (ohne Salt)

```
passwort123 -> MD5 -> 482c811da5d5b4bc6d497ffa98491e38
passwort123 -> MD5 -> 482c811da5d5b4bc6d497ffa98491e38   (identisch, jedes Mal)
```

Gut zu wissen

Genau diese Eigenschaft – derselbe Klartext ergibt immer denselben Hash – macht sogenannte **Rainbow Tables** möglich: riesige, vorab berechnete Listen von Klartext-Hash-Paaren. Ein Treffer in einer solchen Tabelle verrät das Passwort, ohne dass überhaupt gerechnet werden muss.

2.2 Salting als Gegenmaßnahme

Ein **Salt** ist eine zufällige, pro Passwort individuelle Zeichenfolge, die vor dem Hashen mit dem Passwort verbunden wird.

Beispiel 2.2: Derselbe Klartext mit unterschiedlichem Salt

```
passwort123 + Salt "a1b2" -> Hash X
passwort123 + Salt "c3d4" -> Hash Y   (komplett anderer Hash, trotz gleichem Passwort)
```

Mit Salting führt derselbe Klartext zu unterschiedlichen Hashes – vorberechnete Rainbow Tables werden dadurch wirkungslos, weil für jeden individuellen Salt neu gerechnet werden müsste. Moderne Systeme speichern den Salt zusammen mit dem Hash (er muss ja für die spätere Prüfung bekannt sein), aber eben **individuell pro Nutzerkonto**.

Verfahren	Salting	Rechenaufwand pro Versuch	Cracking-Geschwindigkeit
unsalted MD5	nein	sehr gering	extrem schnell

Verfahren	Salting	Rechenaufwand pro Versuch	Cracking-Geschwindigkeit
gesaltetes SHA-256	ja	gering	schnell, aber langsamer als unsalted
bcrypt/scrypt/Argon2	ja, eingebaut	absichtlich hoch	deutlich langsamer, oft um Größenordnungen

> **Gut zu wissen**
>
> bcrypt, scrypt und Argon2 sind absichtlich **rechenintensiv** gestaltet (über einen einstellbaren „Kostenfaktor"), gerade um Cracking-Versuche massiv zu verlangsamen. Das ist der Hauptgrund, warum moderne Passwort-Speicherung diese Verfahren gegenüber einfachem, auch gesaltetem SHA-256 bevorzugt.

2.3 Warum „Cracken" eigentlich Raten und Vergleichen ist

Ein Hash lässt sich nicht umkehren. John the Ripper „knackt" einen Hash deshalb nicht im Sinne eines Entschlüsselns, sondern durch **systematisches Ausprobieren**: Es nimmt einen Kandidaten (aus einer Wortliste, einer Regel oder einer vollständigen Kombination), hasht ihn mit demselben Verfahren und demselben Salt, und vergleicht das Ergebnis mit dem Ziel-Hash.

Beispiel 2.3: Das Grundprinzip in Pseudocode

```
für jeden Kandidaten aus Wortliste/Regeln/Kombinationen:
    kandidat_hash = hash(kandidat, salt)
    wenn kandidat_hash == ziel_hash:
        Treffer! Kandidat ist das ursprüngliche Passwort
```

Die Geschwindigkeit dieses Prozesses hängt vollständig vom verwendeten Hash-Verfahren ab (Tabelle oben) – das ist der Kern dessen, was in Kapitel 8 als Performance-Thema vertieft wird.

Zusammenfassung

> **Zusammenfassung**
>
> - Systeme speichern Passwort-Hashes statt Klartext; die Anmeldung vergleicht neu berechnete mit gespeicherten Hashes.
> - Salting macht denselben Klartext zu unterschiedlichen Hashes und entwertet vorberechnete Rainbow Tables.
> - bcrypt, scrypt und Argon2 sind absichtlich rechenintensiv und verlangsamen Cracking-Versuche massiv.
> - „Cracken" bedeutet systematisches Ausprobieren und Vergleichen, nicht das Umkehren eines Hashes.

Kapitel 3: Installation und Laborumgebung

Lernziele

- Du installierst John the Ripper (Jumbo-Version) unter Kali, Debian/Ubuntu und Fedora/Arch.
- Du erzeugst dir eigene, harmlose Testhashes für alle folgenden Übungen.
- Du richtest eine passende Wortliste für dein Labor ein.

3.1 Installation

Beispiel 3.1: John the Ripper installieren

```
# Kali Linux: Jumbo-Version bereits vorinstalliert
$ john --version

# Debian/Ubuntu
$ sudo apt update && sudo apt install john

# Fedora
$ sudo dnf install john
# Arch Linux
$ sudo pacman -S john
```

Gut zu wissen

Die über die Paketverwaltung installierte Version ist nicht immer die vollständige **Jumbo-Version**. Prüfe mit `john --list=formats | wc -l`, wie viele Hash-Formate deine Installation unterstützt – die Jumbo-Version deckt üblicherweise mehrere hundert Formate ab, die Basisversion deutlich weniger. Bei Bedarf lässt sich die Jumbo-Version aus dem Quellcode des offiziellen Projekts bauen.

3.2 Eigene Testhashes erzeugen

Beispiel 3.2: Testhashes aus frei erfundenen Passwörtern erzeugen

```
$ echo -n "testpass123" | sha256sum
$ echo -n "testpass123" | md5sum
$ python3 -c "import crypt; print(crypt.crypt('testpass123', \
    crypt.mksalt(crypt.METHOD_SHA512)))"
```

TIPP – Profi-Trick

Lege dir für die Übungen eine eigene Datei `testhashes.txt` mit mehreren selbst erzeugten Hashes aus **frei erfundenen** Testpasswörtern an. So brauchst du für kein einziges Beispiel in diesem Buch echte Zugangsdaten.

3.3 Eine Testwortliste bereitstellen

Beispiel 3.3: Eine kleine, eigene Wortliste anlegen

```
$ cat > testliste.txt << 'EOF'
passwort123
testpass123
sommer2026
qwertz123
EOF
```

Gut zu wissen

Die bekannte Wortliste **rockyou.txt** (Millionen real geleakter, inzwischen jahrealter Passwörter) liegt in Kali vorinstalliert unter `/usr/share/wordlists/rockyou.txt.gz` und muss vor der Nutzung entpackt werden (`gunzip`). Für die Übungen in diesem Buch reicht aber zunächst eine kleine, selbst erstellte Liste völlig aus – sie macht die Abläufe nachvollziehbarer, bevor du mit einer großen Liste arbeitest.

3.4 Laborumgebung

Für dieses Buch genügt deine Kali-VM allein: John the Ripper arbeitet ausschließlich mit lokal vorliegenden Hash-Dateien, es ist kein zweites Zielsystem und kein Netzwerkzugriff nötig.

LEGAL – nur mit Auftrag/Einwilligung

Auch ohne Netzwerkkomponente gilt die Grundregel dieser Reihe unverändert: Nutze ausschließlich selbst erzeugte Testhashes oder ausdrücklich zu Übungszwecken bereitgestellte Dateien (Kapitel 1.4 und 10).

Zusammenfassung

Zusammenfassung

- Die Jumbo-Version deckt deutlich mehr Hash-Formate ab als die Basisversion; `--list=formats` zeigt den Unterschied.
- Eigene Testhashes lassen sich mit `sha256sum`, `md5sum` oder Pythons `crypt`-Modul aus frei erfundenen Passwörtern erzeugen.
- Eine kleine, selbst erstellte Wortliste reicht für den Einstieg; `rockyou.txt` steht in Kali für spätere Übungen bereit.
- Für dieses Buch genügt die Kali-VM allein, ohne Netzwerkkomponente.

Kapitel 4: Angriffsmodi

Lernziele

- Du kennst die drei grundlegenden Angriffsmodi von John the Ripper.
- Du verstehst, in welcher Reihenfolge John sie standardmäßig durchläuft.
- Du kannst einen Modus gezielt erzwingen.

4.1 Die drei Grundmodi im Überblick

Modus	Option	Grundidee
Single Crack	`--single`	nutzt Informationen aus der Hash-Datei selbst (z. B. Benutzernamen) als Kandidaten
Wortliste	`--wordlist=<datei>`	probiert jeden Eintrag einer Wortliste, optional mit Regeln kombiniert (Kapitel 6)
Incremental	`--incremental`	probiert systematisch alle Zeichenkombinationen (Kapitel 7)

Beispiel 4.1: Ohne Angabe eines Modus – Johns Standardreihenfolge

```
$ john testhashes.txt
```

Ohne explizite Modusangabe durchläuft John standardmäßig zuerst **Single Crack** (schnell, wenig Kandidaten), dann eine **Wortliste** mit Standardregeln, und zuletzt bei Bedarf den **Incremental-Modus** – vom Schnellsten und Wahrscheinlichsten zum Aufwendigsten.

4.2 Single Crack

Beispiel 4.2: Single-Crack-Modus gezielt einsetzen

```
$ john --single --format=sha256crypt testhashes.txt
```

Dieser Modus nutzt Kontextinformationen aus der Hash-Datei selbst – etwa den Benutzernamen `admin` als Kandidaten `admin`, `Admin`, `admin123` und ähnliche Varianten. Er ist besonders schnell und findet erstaunlich oft Treffer bei Konten, deren Passwort auf dem eigenen Benutzernamen basiert.

4.3 Wortlisten-Modus

Beispiel 4.3: Wortlisten-Modus mit eigener Testliste

```
$ john --wordlist=testliste.txt testhashes.txt
Loaded 3 password hashes with 3 different salts
testpass123      (eugen)
1g 0:00:00:00 DONE
```

Beispiel 4.4: Ergebnisse ansehen

```
$ john --show testhashes.txt
eugen:testpass123
```

`--show` zeigt bereits gefundene Passwörter an, ohne den Cracking-Vorgang erneut zu starten – die Ergebnisse werden dafür in der sogenannten **Potfile** gespeichert (Kapitel 8.3).

4.4 Incremental-Modus (kurzer Vorgriff)

Beispiel 4.5: Incremental-Modus, zeitlich begrenzt

```
$ john --incremental --format=raw-md5 -max-run-time=30 testhashes.txt
```

Der Incremental-Modus probiert systematisch alle möglichen Zeichenkombinationen innerhalb eines definierten Zeichensatzes und einer Längenbeschränkung – ohne zeitliche Begrenzung kann das je nach Hash-Verfahren extrem lange dauern. Kapitel 7 vertieft diesen Modus und die verwandten, gezielteren **Masken**.

> **TIPP – Profi-Trick**
>
> `-max-run-time=<sekunden>` begrenzt jeden Lauf zeitlich und ist besonders beim Ausprobieren des Incremental-Modus sinnvoll, damit ein Testlauf nicht versehentlich stundenlang weiterläuft.

Zusammenfassung

> **Zusammenfassung**
>
> - Single Crack, Wortliste und Incremental sind die drei Grundmodi, die John standardmäßig in dieser Reihenfolge durchläuft.
> - Single Crack nutzt Kontextinformationen wie Benutzernamen und ist besonders schnell.
> - Der Wortlisten-Modus ist der am häufigsten genutzte Modus im Alltag; `--show` liest bereits gefundene Passwörter aus der Potfile.
> - Der Incremental-Modus probiert systematisch alle Kombinationen und sollte zeitlich begrenzt werden.

Kapitel 5: Hash-Formate erkennen und extrahieren

Lernziele

- Du erkennst gängige Hash-Formate an ihrer Struktur.
- Du extrahierst Hashes aus typischen Dateiformaten mit den *2john-Werkzeugen.
- Du gibst das richtige Format explizit an, statt dich allein auf die Erkennung zu verlassen.

5.1 Hash-Formate an der Struktur erkennen

Erscheinungsbild (Beispiel)	Wahrscheinliches Format
`482c811da5d5b4bc6d497ffa98491e38` (32 Zeichen hex)	MD5
`$1$abcd1234$...`	MD5-Crypt (klassisches Unix-Format)
`$6$abcd1234$...`	SHA-512-Crypt (modernes Unix-Format, lang)
`$2a$` / `$2b$` / `$2y$` am Anfang	bcrypt
`5f4dcc3b5aa765d61d8327deb882cf99` (identisch wirkende 32-Zeichen-Hex-Werte für dasselbe Passwort)	wahrscheinlich unsalted MD5

Beispiel 5.1: Das Format explizit angeben, statt zu raten

```
$ john --format=raw-md5 testhashes.txt
$ john --format=sha512crypt testhashes.txt
```

WARNUNG – Sicherheitsrisiko

Verlasse dich bei wichtigen Prüfungen nicht allein auf Johns automatische Formaterkennung. Manche Hash-Formate sehen einander strukturell ähnlich, werden aber unterschiedlich berechnet – eine falsche Formatannahme führt dazu, dass selbst das richtige Passwort nie als Treffer erkannt wird. `--list=formats` zeigt dir alle unterstützten Bezeichnungen.

5.2 Hashes aus Linux-Systemdateien extrahieren

Beispiel 5.2: /etc/shadow und /etc/passwd zu einem John-Format kombinieren

```
$ sudo unshadow /etc/passwd /etc/shadow > kombiniert.txt
$ john kombiniert.txt
```

`unshadow` fasst die öffentliche Kontoinformation aus `/etc/passwd` mit den eigentlichen Hashes aus `/etc/shadow` zu einer Datei zusammen, wie John sie erwartet.

ILLEGAL – niemals ohne Auftrag

Führe `unshadow` ausschließlich auf einem System aus, dessen `/etc/shadow` du mit eigener, autorisierter Administratorberechtigung einsehen darfst – auf deiner eigenen Test-VM oder im Rahmen eines autorisierten Audits. `/etc/shadow` enthält die Passwort-Hashes aller Systemkonten; unbefugter Zugriff darauf ist bereits für sich genommen ein Verstoß gegen § 202a StGB.

5.3 Hashes aus Dateien extrahieren: die *2john-Werkzeuge

Die Jumbo-Version bringt zahlreiche spezialisierte Werkzeuge mit, die den Passwort-Hash aus verschlüsselten Dateien in ein von John lesbares Format extrahieren.

Beispiel 5.3: Hash aus einer passwortgeschützten ZIP-Datei extrahieren

```
$ zip2john test.zip > zip_hash.txt
$ john zip_hash.txt
```

Beispiel 5.4: Weitere gängige *2john-Werkzeuge

```
$ pdf2john.pl geschuetzt.pdf > pdf_hash.txt
$ ssh2john.py id_rsa > ssh_hash.txt
$ office2john.py dokument.docx > office_hash.txt
```

TIPP – Profi-Trick

Eine vollständige Liste aller verfügbaren *2john-Werkzeuge findest du mit `ls /usr/share/john/` oder, je nach Installation, `ls /usr/sbin/2john`. Für jedes gängige Dateiformat mit Passwortschutz existiert meist ein passendes Extraktionswerkzeug.

Zusammenfassung

Zusammenfassung

- Hash-Formate lassen sich oft an Präfix und Länge erkennen, sollten bei wichtigen Prüfungen aber explizit angegeben werden.
- `unshadow` kombiniert `/etc/passwd` und `/etc/shadow` zu einem für John lesbaren Format – ausschließlich mit eigener Berechtigung.
- Die *2john-Werkzeuge extrahieren Hashes aus ZIP-, PDF-, SSH-Schlüssel- und Office-Dateien.
- `--list=formats` zeigt alle von der Installation unterstützten Hash-Formate.

Kapitel 6: Regeln – Wortlisten intelligent erweitern

Lernziele

- Du verstehst, warum Regeln eine kleine Wortliste wirkungsvoller machen als reines Durchprobieren.
- Du wendest vordefinierte Regelsätze an.
- Du kannst eine einfache eigene Regel formulieren.

6.1 Warum Regeln?

Menschen variieren Passwörter oft nach vorhersagbaren Mustern: aus `passwort` wird `Passwort1`, `p4ssw0rt` oder `passwort!`. **Regeln** wenden genau solche Transformationen automatisch auf jeden Wortlisteneintrag an, statt dass die Wortliste selbst jede Variante enthalten müsste.

Beispiel 6.1: Wortliste mit Standardregeln kombinieren

```
$ john --wordlist=testliste.txt --rules testhashes.txt
```

Beispiel 6.2: Was Regeln aus einem einzigen Eintrag machen (Konzept)

```
Wortlisteneintrag: passwort
Mit Regeln erzeugte Kandidaten (Auswahl):
  Passwort       (erster Buchstabe groß)
  passwort1      (Ziffer angehängt)
  passwort!      (Sonderzeichen angehängt)
  p4ssw0rt       (Leetspeak-Ersetzung a->4, o->0)
  tro wssap      (umgekehrt, je nach Regelsatz)
```

Gut zu wissen

Aus einer Wortliste mit nur wenigen tausend Einträgen können Regeln mehrere hunderttausend zusätzliche, realistische Kandidaten erzeugen – oft mit deutlich besserer Trefferquote pro Zeiteinheit, als dieselbe Rechenzeit in den vollständig ungerichteten Incremental-Modus (Kapitel 7) zu investieren.

6.2 Vordefinierte Regelsätze

Beispiel 6.3: Einen bestimmten Regelsatz auswählen

```
$ john --wordlist=testliste.txt --rules=jumbo testhashes.txt
$ john --wordlist=testliste.txt --rules=best64 testhashes.txt
```

Regelsatz	Charakteristik
`wordlist` (Standard)	grundlegende, schnelle Transformationen
`jumbo`	umfangreicher Satz, der viele reale Passwortmuster abdeckt
`best64`	ein bekannter, kompakter Satz der 64 wirkungsvollsten Regeln (ursprünglich für Hashcat entwickelt, auch für John verfügbar)

6.3 Eine eigene, einfache Regel

Regeln werden in der Konfigurationsdatei `john.conf` definiert. Eine minimale eigene Regel könnte so aussehen:

Beispiel 6.4: Eine eigene Regel in john.conf ergänzen (Auszug)

```
[List.Rules:Eigene]
c $1 $2 $3
```

Diese Regel verwandelt jeden Wortlisteneintrag in eine großgeschriebene Variante mit angehängten Ziffern `1`, `2`, `3` (z. B. aus `test` wird `Test1`, `Test2`, `Test3`).

Beispiel 6.5: Die eigene Regel verwenden

```
$ john --wordlist=testliste.txt --rules=Eigene testhashes.txt
```

> **TIPP – Profi-Trick**
>
> Die vollständige Regelsyntax (`c` für „ersten Buchstaben groß", `$X` für „Zeichen X anhängen", und viele weitere Funktionen) ist in `john.conf` selbst ausführlich dokumentiert und mit zahlreichen Beispielen versehen – ein Blick hinein lohnt sich, bevor du eigene Regeln von Grund auf schreibst.

Zusammenfassung

> **Zusammenfassung**
>
> - Regeln wenden vorhersagbare menschliche Passwortmuster automatisch auf jeden Wortlisteneintrag an.
> - Vordefinierte Regelsätze wie `jumbo` oder `best64` decken bereits viele reale Muster ab.
> - Eigene Regeln werden in `john.conf` definiert und lassen sich gezielt mit `--rules=<name>` einsetzen.
> - Gute Regeln liefern oft eine bessere Trefferquote pro Zeiteinheit als der ungerichtete Incremental-Modus.

Kapitel 7: Masken und der Incremental-Modus im Detail

Lernziele

- Du verstehst den Unterschied zwischen ungerichtetem Incremental-Modus und gezielten Masken.
- Du formulierst eine eigene Maske für ein bekanntes Passwortmuster.
- Du kannst den Suchraum einer Maske grob abschätzen.

7.1 Incremental-Modus: der vollständige, aber teure Weg

Der **Incremental-Modus** probiert systematisch alle möglichen Zeichenkombinationen innerhalb eines Zeichensatzes und einer Längenbeschränkung – er findet theoretisch jedes Passwort, braucht dafür aber mit wachsender Länge und Zeichensatzgröße exponentiell mehr Zeit.

Beispiel 7.1: Incremental-Modus mit begrenztem Zeichensatz und Länge

```
$ john --incremental=Digits -max-run-time=60 testhashes.txt
```

Beispiel 7.2: Verfügbare Incremental-Modi auflisten

```
$ john --list=inc-modes
```

WARNUNG – Sicherheitsrisiko

Ein uneingeschränkter Incremental-Lauf gegen ein starkes Hash-Verfahren (z. B. bcrypt) über den vollen druckbaren Zeichensatz kann praktisch beliebig lange dauern – Jahre bis Jahrzehnte sind je nach Passwortlänge keine Übertreibung. Begrenze Testläufe immer mit `-max-run-time`, besonders beim Ausprobieren.

7.2 Masken: gezielt statt vollständig

Eine **Maske** beschreibt ein bekanntes oder vermutetes Muster, zum Beispiel „ein großer Buchstabe, gefolgt von sechs Kleinbuchstaben, gefolgt von zwei Ziffern" – deutlich zielgerichteter als ein vollständiger Incremental-Lauf über denselben Zeichensatz.

Beispiel 7.3: Maskenbasierter Angriff

```
$ john --mask='?u?l?l?l?l?l?d?d' testhashes.txt
```

Platzhalter	Bedeutung
`?l`	Kleinbuchstabe
`?u`	Großbuchstabe
`?d`	Ziffer
`?s`	Sonderzeichen
`?a`	beliebiges druckbares Zeichen

Beispiel 7.4: Maske mit festem Präfix kombinieren

```
$ john --mask='Sommer?d?d' testhashes.txt
```

Diese Maske probiert ausschließlich `Sommer00` bis `Sommer99` – nützlich, wenn aus einer Passwort-Richtlinie oder einem Muster in bereits geknackten Passwörtern bekannt ist, dass viele Nutzer:innen saisonale Begriffe mit zweistelligen Zahlen kombinieren.

Gut zu wissen

Der Suchraum einer Maske lässt sich grob abschätzen: `?u?l?l?l?l?l?l?d?d` hat 26 Möglichkeiten für `?u`, je 26 für jedes `?l` (sechsmal) und je 10 für jedes `?d` (zweimal), also $26 \times 26^6 \times 10^2 \approx 2 \times 10^{11}$ Kombinationen. Je kürzer und spezifischer die Maske, desto realistischer wird ein vollständiger Durchlauf in angemessener Zeit.

7.3 Masken aus bereits geknackten Passwörtern ableiten

TIPP – Profi-Trick

Ein bewährter Ablauf: Knacke zunächst mit Wortliste und Regeln (Kapitel 6) so viele Hashes wie möglich, sieh dir die gefundenen Passwörter mit `john --show` an, und leite daraus wiederkehrende Muster für eine gezielte Maske gegen die verbleibenden, noch ungelösten Hashes ab.

Zusammenfassung

Zusammenfassung

- Der Incremental-Modus probiert vollständig, aber mit exponentiell wachsendem Zeitaufwand bei längeren Passwörtern.
- Masken (`?l`, `?u`, `?d`, `?s`, `?a`) zielen auf ein bekanntes oder vermutetes Muster und sind dadurch deutlich effizienter.
- Der Suchraum einer Maske lässt sich grob über die Anzahl möglicher Zeichen je Platzhalter abschätzen.
- Bereits geknackte Passwörter liefern oft die besten Hinweise für eine gezielte Maske gegen die verbleibenden Hashes.

Kapitel 8: Performance, Sessions und die Potfile-Verwaltung

Lernziele

- Du verstehst, welche Faktoren die Cracking-Geschwindigkeit bestimmen.
- Du unterbrichst und setzt lange Läufe gezielt fort.
- Du verwaltest die Potfile bewusst, statt sie zu übersehen.

8.1 Was die Geschwindigkeit bestimmt

Beispiel 8.1: Geschwindigkeit verschiedener Hash-Verfahren benchmarken

```
$ john --test --format=raw-md5
$ john --test --format=sha512crypt
$ john --test --format=bcrypt
```

Die Ausgabe zeigt die Anzahl der Hash-Berechnungen pro Sekunde auf deiner Hardware. Der Unterschied zwischen einem schnellen, unsalted Verfahren und einem absichtlich langsamen Verfahren wie bcrypt beträgt oft mehrere Größenordnungen – der praktische Nachweis für das, was Kapitel 2.2 bereits in der Theorie beschreibt.

Faktor	Einfluss
Hash-Verfahren	größter Einflussfaktor, von Faktor 1 bis zu mehreren tausendfach
CPU-Kerne (`--fork`)	mehrere Kerne parallel nutzen
GPU-Unterstützung	für manche Formate über OpenCL deutlich schneller als CPU allein
Länge und Zeichensatz der Kandidaten	insbesondere bei Incremental-Modus und Masken entscheidend

Beispiel 8.2: Mehrere CPU-Kerne nutzen

```
$ john --fork=4 --wordlist=testliste.txt testhashes.txt
```

8.2 Sessions: lange Läufe unterbrechen und fortsetzen

Beispiel 8.3: Einen benannten Lauf starten

```
$ john --session=testlauf --wordlist=testliste.txt --rules testhashes.txt
```

Mit `Strg+C` lässt sich ein laufender Vorgang jederzeit unterbrechen, ohne den Fortschritt zu verlieren.

Beispiel 8.4: Einen unterbrochenen Lauf fortsetzen

```
$ john --restore=testlauf
```

TIPP – Profi-Trick

Vergib für jeden ernsthafteren Lauf einen eigenen `--session`-Namen. So kannst du mehrere Vorgänge parallel verfolgen und gezielt genau den richtigen fortsetzen, statt dich auf den zuletzt unterbrochenen Standardlauf verlassen zu müssen.

8.3 Die Potfile

Jedes gefundene Passwort landet automatisch in der **Potfile** (standardmäßig `~/.john/john.pot`), unabhängig davon, in welchem Modus oder Lauf es gefunden wurde.

Beispiel 8.5: Mit der Potfile arbeiten

```
$ john --show testhashes.txt        # bereits gefundene Passwörter anzeigen
$ cat ~/.john/john.pot               # die Potfile direkt einsehen
```

WARNUNG – Sicherheitsrisiko

Die Potfile sammelt über die Zeit **alle** jemals geknackten Passwörter an, auch aus früheren, längst vergessenen Projekten. Da sie reale, sensible Zugangsdaten im Klartext enthalten kann, gehört sie ebenso geschützt wie jeder andere Fund aus einem autorisierten Test (Ethical-Hacking-Handbuch, Kapitel 8.5) – verschlüsselt speichern, nicht unnötig lange aufbewahren, nicht ungeschützt weitergeben.

Zusammenfassung

Zusammenfassung

- Das Hash-Verfahren ist der mit Abstand größte Einflussfaktor auf die Cracking-Geschwindigkeit, messbar mit `--test`.
- `--fork` nutzt mehrere CPU-Kerne parallel; für einzelne Formate steht zusätzlich GPU-Unterstützung zur Verfügung.
- `--session` und `--restore` erlauben, lange Läufe gezielt zu unterbrechen und fortzusetzen.
- Die Potfile sammelt alle jemals gefundenen Passwörter und muss entsprechend sorgfältig behandelt werden.

Kapitel 9: John the Ripper und Hashcat im Vergleich

Lernziele

- Du kennst die wichtigsten Unterschiede zwischen John the Ripper und Hashcat.
- Du kannst begründen, wann welches Werkzeug die bessere Wahl ist.
- Du verstehst, dass beide dieselben Grundkonzepte (Wortlisten, Regeln, Masken) nutzen.

9.1 Zwei verwandte, aber unterschiedlich ausgerichtete Werkzeuge

Das Ethical-Hacking-Handbuch dieser Reihe erwähnt in Kapitel 7.1 bereits **Hashcat** als Alternative zu John the Ripper. Beide lösen dieselbe Grundaufgabe – systematisches Ausprobieren und Vergleichen von Passwort-Kandidaten (Kapitel 2.3) –, unterscheiden sich aber in Schwerpunkt und Architektur.

Merkmal	John the Ripper	Hashcat
Ursprünglicher Schwerpunkt	CPU-basiertes Cracking, Unix-Passwortformate	GPU-beschleunigtes Cracking
Formatvielfalt	sehr breit, besonders in der Jumbo-Version	ebenfalls sehr breit, oft als erstes bei neuen Formaten
Geschwindigkeit bei GPU-geeigneten Formaten	gut, aber meist langsamer als Hashcat	in der Regel schneller, da von Grund auf GPU-zentriert
Bedienkonzept	ein Werkzeug mit vielen Modi und Regeln	ein Werkzeug mit explizit benannten Angriffsmodi (`-a 0` Wortliste, `-a 3` Maske, u. a.)
Typischer Verbreitungsgrad	Standard auf praktisch jedem Pentest-System	ebenfalls Standard, besonders bei verfügbarer GPU

9.2 Dieselben Konzepte, andere Syntax

Beispiel 9.1: Wortlisten-Angriff in beiden Werkzeugen (Konzept, Syntax vereinfacht)

```
John:    john --wordlist=testliste.txt --rules testhashes.txt
Hashcat: hashcat -a 0 -m <modus> testhashes.txt testliste.txt
```

Beispiel 9.2: Maskenbasierter Angriff in beiden Werkzeugen (Konzept)

```
John:    john --mask='?u?l?l?l?l?l?l?d?d' testhashes.txt
Hashcat: hashcat -a 3 -m <modus> testhashes.txt '?u?l?l?l?l?l?l?d?d'
```

Gut zu wissen

Die Maskensyntax (`?l`, `?u`, `?d`, `?s`) ist zwischen beiden Werkzeugen praktisch identisch – wer eine Maske für John formuliert hat, kann sie mit minimalen Anpassungen auch in Hashcat einsetzen, und umgekehrt.

9.3 Wann welches Werkzeug?

Situation	Empfehlung
Keine oder schwache GPU verfügbar	John the Ripper, da von Anfang an CPU-zentriert optimiert
Starke GPU vorhanden, viele Hashes	Hashcat, meist deutlich schneller
Unix-Systemdateien (`/etc/shadow`) direkt prüfen	John the Ripper, mit `unshadow` besonders eingespielt
Ein ungewöhnliches, sehr neues Hash-Format	beide prüfen – je nach Format kann die Unterstützung unterschiedlich aktuell sein
Lernen der Grundkonzepte	mit John beginnen: etwas einfachere Bedienung für den Einstieg

TIPP – Profi-Trick

Du musst dich nicht für eines der beiden Werkzeuge entscheiden. In der Praxis haben viele Pentester:innen beide installiert und wählen je nach verfügbarer Hardware und Hash-Format situativ aus.

Zusammenfassung

Zusammenfassung

- John the Ripper und Hashcat lösen dieselbe Grundaufgabe mit unterschiedlichem historischem Schwerpunkt: CPU- versus GPU-zentriert.
- Wortlisten-, Regel- und Maskenkonzepte sind zwischen beiden Werkzeugen weitgehend übertragbar.
- Bei starker GPU und großen Hash-Mengen ist Hashcat meist schneller; John the Ripper punktet bei CPU-only-Umgebungen und Unix-Systemdateien.
- Viele Pentester:innen nutzen situativ beide Werkzeuge nebeneinander.

Kapitel 10: Praxisteil – Laboraufbau und geführte Übungen

Lernziele

- Du baust das für dieses Buch nötige Labor auf.
- Du arbeitest acht Übungen durch, die alle vorherigen Kapitel praktisch verbinden.
- Du weißt, dass die Laufzeit einzelner Übungen stark von deiner Hardware abhängt, nicht automatisch ein Fehler ist.

10.1 Laboraufbau

Für dieses Buch genügt eine einzelne Kali-VM mit installiertem John the Ripper (Kapitel 3). Ein zweites Zielsystem ist nicht nötig, da ausschließlich mit lokal erzeugten Testhashes gearbeitet wird.

10.2 Checkliste: sicherer Umgang mit Testdaten

Checkliste 10.1: Vor jeder Übung

```
[ ] Alle verwendeten Hashes stammen aus selbst erzeugten, frei erfundenen
    Testpasswörtern (Kapitel 3.2) – keine echten Zugangsdaten
[ ] Bei Übung 5 (unshadow): nur auf einer eigenen Test-VM mit eigenem,
    administrativem Zugriff ausgeführt
[ ] Lange Läufe (Incremental, große Masken) mit -max-run-time begrenzt
[ ] Potfile und Testhashes nach Abschluss der Übungen bei Bedarf gelöscht
```

10.3 Übungen

Jede Übung folgt demselben Schema: Ziel, Voraussetzungen, Durchführung, erwartetes Ergebnis, Auswertung und Fehlerbehebung.

Übung 1: Eigene Testhashes erzeugen und mit Single Crack prüfen

Ziel: Den Single-Crack-Modus und seine Nutzung von Kontextinformationen erleben. **Voraussetzungen:** Kali-VM mit John the Ripper. **Durchführung:** Erzeuge einen Hash für das Passwort `eugen123` mit Benutzernamen `eugen` (Beispiel 3.2), speichere ihn im Format `benutzername:hash` in `testhashes.txt`. Führe `john --single testhashes.txt` aus (Beispiel 4.2). **Erwartetes Ergebnis:** John findet das Passwort innerhalb weniger Sekunden, weil es den Benutzernamen als Kandidatenbasis nutzt. **Auswertung:** Prüfe mit `john --show testhashes.txt`, ob das Passwort korrekt angezeigt wird. **Fehlerbehebung:** Kein Treffer? Prüfe mit `--format=<name>`, ob John das Hash-Format korrekt erkannt hat (Kapitel 5.1).

Übung 2: Wortlisten-Angriff mit und ohne Regeln vergleichen

Ziel: Die Wirkung von Regeln auf eine kleine Wortliste messen. **Voraussetzungen:** Testhashes mit einem Passwort wie `Sommer2026!`, das nicht wörtlich in deiner Testliste steht, aber auf `sommer` basiert. **Durchführung:** Führe zunächst `john --wordlist=testliste.txt testhashes.txt` ohne Regeln aus, danach mit `--rules` (Beispiel 6.1) erneut gegen denselben, zurückgesetzten Hash. **Erwartetes Ergebnis:** Ohne Regeln kein Treffer, mit Regeln ein Treffer (je nach gewähltem Regelsatz). **Auswertung:** Das zeigt den praktischen Nutzen aus Kapitel 6.1: Regeln erschließen Varianten, die in der reinen Wortliste fehlen. **Fehlerbehebung:** Setze vor dem zweiten Lauf die Potfile zurück oder nutze einen neuen Hash, sonst meldet John das Passwort fälschlich sofort aus dem Cache als bereits bekannt.

Übung 3: Geschwindigkeit verschiedener Hash-Verfahren benchmarken

Ziel: Den Geschwindigkeitsunterschied aus Kapitel 2.2 selbst messen. **Voraussetzungen:** wie Übung 1. **Durchführung:** Führe `john --test --format=raw-md5` und `john --test --format=sha512crypt` aus (Beispiel 8.1). Falls verfügbar, zusätzlich `--format=bcrypt`. **Erwartetes Ergebnis:** Die Hashes pro Sekunde unterscheiden sich um ein Vielfaches, oft um mehrere Größenordnungen. **Auswertung:** Notiere die drei Werte und ordne sie der Tabelle in Kapitel 2.2 zu. **Fehlerbehebung:** Meldet John ein Format als nicht unterstützt, prüfe mit `--list=formats`, ob deine Installation die Jumbo-Version ist (Kapitel 3.1).

Übung 4: Eine gezielte Maske einsetzen

Ziel: Ein bekanntes Passwortmuster effizienter prüfen als mit vollständigem Incremental-Modus. **Voraussetzungen:** ein Testhash für ein Passwort nach dem Muster „Großbuchstabe + sechs Kleinbuchstaben + zwei Ziffern", z. B. `Muster42`. **Durchführung:** Führe `john --mask='?u?l?l?l?l?l?d?d'` gegen diesen Hash aus (Beispiel 7.3). **Erwartetes Ergebnis:** Ein Treffer innerhalb überschaubarer Zeit, da der Suchraum deutlich kleiner ist als ein vollständiger Incremental-Lauf über denselben Zeichensatz. **Auswertung:** Schätze den Suchraum nach der Formel aus Kapitel 7.2 ab und vergleiche ihn mit der tatsächlich gemessenen Laufzeit. **Fehlerbehebung:** Dauert der Lauf ungewöhnlich lange, prüfe, ob die Maske tatsächlich zur Länge deines Testpassworts passt.

Übung 5: Eigene /etc/shadow-Hashes extrahieren (nur eigene VM)

Ziel: Den `unshadow`-Workflow an einem System nachvollziehen, das dir vollständig gehört. **Voraussetzungen:** administrativer Zugriff auf deine eigene Test-VM. **Durchführung:** Lege auf der Test-VM ein neues Testkonto mit einem frei erfundenen Passwort an, führe `sudo unshadow /etc/passwd /etc/shadow > kombiniert.txt` aus (Beispiel 5.2) und knacke anschließend dieses eine, selbst angelegte Testkonto. **Erwartetes Ergebnis:** Dein selbst gesetztes Testpasswort wird gefunden. **Auswertung:** Dieser Ablauf zeigt, wie ein autorisiertes Post-Exploitation-Audit (Ethical-Hacking-Handbuch, Kapitel 7) in der Praxis aussieht. **Fehlerbehebung:** Lösche das angelegte Testkonto nach der Übung wieder, und bewahre `kombiniert.txt` nicht unnötig auf (Kapitel 8.3).

Übung 6: Hash aus einer eigenen ZIP-Datei extrahieren

Ziel: Ein *2john-Werkzeug praktisch einsetzen. **Voraussetzungen:** eine selbst erstellte, passwortgeschützte ZIP-Datei mit frei erfundenem Passwort. **Durchführung:** Führe `zip2john test.zip > zip_hash.txt` aus, danach `john zip_hash.txt` (Beispiel 5.3). **Erwartetes Ergebnis:** Das selbst gewählte ZIP-Passwort wird gefunden. **Auswertung:** Vergleiche die Zeit bis zum Treffer mit Übung 1 – ZIP-Hashes sind je nach Verschlüsselungsmethode unterschiedlich aufwendig zu prüfen. **Fehlerbehebung:** Meldet `zip2john` einen Fehler, prüfe, ob die ZIP-Datei tatsächlich mit einem klassischen Passwortschutz (nicht nur einem Archivkommentar) erstellt wurde.

Übung 7: Einen Lauf unterbrechen und fortsetzen

Ziel: Session-Verwaltung praktisch nutzen. **Voraussetzungen:** ein Testhash mit einem absichtlich schwer zu findenden Passwort (z. B. 8 zufällige Zeichen) und einem entsprechend lange laufenden Incremental-Modus. **Durchführung:** Starte `john --session=testlauf --incremental -max-run-time=20` (Beispiel 8.3), unterbrich mit `Strg+C`, setze danach mit `john --restore=testlauf` fort. **Erwartetes Ergebnis:** Der Lauf setzt dort fort, wo er unterbrochen wurde, statt von vorne zu beginnen. **Auswertung:** Vergleiche die Gesamtlaufzeit beider Teile mit einem ununterbrochenen Lauf gleicher Gesamtdauer. **Fehlerbehebung:** Meldet `--restore` einen Fehler, prüfe mit `john --status=testlauf`, ob die Session tatsächlich noch existiert.

Übung 8: John the Ripper und Hashcat-Syntax gegenüberstellen

Ziel: Die konzeptionelle Nähe beider Werkzeuge nachvollziehen (sofern Hashcat installiert ist). **Voraussetzungen:** Hashcat zusätzlich installiert (optional). **Durchführung:** Formuliere für denselben Testhash aus Übung 2 sowohl den John- als auch den entsprechenden Hashcat-Aufruf (Beispiel 9.1) und führe beide aus. **Erwartetes Ergebnis:** Beide Werkzeuge finden dasselbe Passwort, möglicherweise mit unterschiedlicher Geschwindigkeit. **Auswertung:** Ordne die beobachteten Laufzeitunterschiede der Tabelle in Kapitel 9.1 zu. **Fehlerbehebung:** Ist Hashcat nicht verfügbar, überspringe den praktischen Teil und vergleiche die Befehle nur schriftlich.

Zusammenfassung

Zusammenfassung

- Für dieses Buch genügt eine einzelne Kali-VM; alle Übungen arbeiten mit selbst erzeugten Testhashes.
- Die Übungen verbinden Angriffsmodi, Hash-Extraktion, Regeln, Masken, Performance und den Vergleich mit Hashcat.
- Laufzeiten hängen stark von der eigenen Hardware ab; längere Läufe sind kein Fehler, sondern ein erwartetes Ergebnis schwächerer Hardware oder anspruchsvollerer Hash-Verfahren.

Kapitel 11: Troubleshooting, Spickzettel, Glossar und Ressourcen

Lernziele

- Du kannst die häufigsten John-the-Ripper-Probleme einordnen und beheben.
- Du hast einen kompakten Spickzettel für den Alltag zur Hand.
- Du kennst weiterführende Ressourcen für die Vertiefung.

11.1 Häufige Probleme

Meldung / Symptom	Wahrscheinliche Ursache	Lösung
„No password hashes loaded"	falsches Dateiformat oder leere Datei	Dateiinhalt mit `cat` prüfen, Format mit `--format` explizit angeben
Passwort wird trotz richtigem Kandidaten nicht gefunden	falsches Hash-Format angenommen	`--list=formats` prüfen, Format explizit statt automatisch erkannt setzen
„Already cracked" ohne erkennbaren Grund	Treffer steckt bereits in der Potfile aus einem früheren Lauf	`john --show` statt erneutem Lauf nutzen, oder Potfile gezielt zurücksetzen
Incremental-Modus läuft „ewig"	zu großer Zeichensatz/Länge ohne Begrenzung	`-max-run-time` setzen, stattdessen Maske (Kapitel 7) oder Regeln (Kapitel 6) erwägen
`--restore` schlägt fehl	Session-Name falsch oder Session bereits abgeschlossen	`john --status=<name>` prüfen, exakten Namen aus `--session` verwenden
*2john-Werkzeug nicht gefunden	Basisversion statt Jumbo-Version installiert	Jumbo-Version installieren (Kapitel 3.1)

11.2 Spickzettel

Beispiel 11.1: Die wichtigsten John-the-Ripper-Befehle auf einen Blick

```
# Verfügbare Formate und Incremental-Modi auflisten
john --list=formats
john --list=inc-modes

# Single Crack, Wortliste, Regeln, Incremental
john --single testhashes.txt
john --wordlist=liste.txt testhashes.txt
john --wordlist=liste.txt --rules testhashes.txt
john --incremental -max-run-time=60 testhashes.txt

# Maskenbasierter Angriff
john --mask='?u?l?l?l?l?l?d?d' testhashes.txt

# Ergebnisse ansehen
john --show testhashes.txt
# Session-Verwaltung
john --session=lauf1 --wordlist=liste.txt testhashes.txt
john --restore=lauf1
john --status=lauf1

# Geschwindigkeit testen
john --test --format=<format>
# Hashes extrahieren
sudo unshadow /etc/passwd /etc/shadow > kombiniert.txt
zip2john datei.zip > hash.txt
```

11.3 Glossar

- **Incremental-Modus** – vollständiges, systematisches Durchprobieren aller Zeichenkombinationen innerhalb eines Zeichensatzes und einer Längenbeschränkung.
- **Jumbo-Version** – die von der Community erweiterte Fassung von John the Ripper mit deutlich mehr Hash-Formaten und Zusatzwerkzeugen.
- **Maske** – ein gezielt formuliertes Muster (z. B. `?u?l?l?d?d`) für einen effizienteren Angriff als der vollständige Incremental-Modus.
- **Potfile** – die Datei, in der John the Ripper alle jemals gefundenen Passwörter sammelt.
- **Rainbow Table** – eine vorab berechnete Liste von Klartext-Hash-Paaren, durch Salting wirkungslos gemacht.
- **Regel (Rule)** – eine automatische Transformation eines Wortlisteneintrags (z. B. Großschreibung, angehängte Ziffer).
- **Salt** – eine zufällige, individuelle Zeichenfolge, die vor dem Hashen mit einem Passwort verbunden wird.
- **Single Crack** – ein Angriffsmodus, der Kontextinformationen aus der Hash-Datei selbst als Kandidaten nutzt.
- **unshadow** – ein Werkzeug, das `/etc/passwd` und `/etc/shadow` zu einem für John lesbaren Format kombiniert.

11.4 Weiterführende Ressourcen

Ressource	Inhalt
Offizielle John-the-Ripper-Dokumentation (openwall.com/john)	vollständige, verbindliche Referenz, inklusive `john.conf`-Regelsyntax
Ethical-Hacking-Handbuch (dieselbe Buchreihe)	Gesamtzusammenhang: Online- vs. Offline-Passwortangriffe, Reporting
hping3- und Nmap-Handbuch (dieselbe Buchreihe)	Netzwerkseite der Reconnaissance und Exploitation
Hashcat-Dokumentation (hashcat.net)	ergänzendes, GPU-zentriertes Cracking-Werkzeug (Kapitel 9)
TryHackMe / HackTheBox	legale Übungsplattformen mit eigenen Passwort-Cracking-Räumen

TIPP – Profi-Trick

Die Datei `john.conf` lohnt einen eigenen, ausführlichen Blick: Sie enthält nicht nur die Regelsyntax, sondern auch Beispielkonfigurationen für Incremental-Modi und viele auskommentierte, lehrreiche Beispiele.

Zusammenfassung

Zusammenfassung

- Die häufigsten Probleme betreffen falsche Formatannahmen, die Potfile und unbegrenzt laufende Incremental-Läufe.
- Der Spickzettel deckt die wichtigsten Befehlsmuster aller vorherigen Kapitel ab.
- Die offizielle Dokumentation, das Ethical-Hacking-Handbuch und die Hashcat-Dokumentation sind die wichtigsten Anschlussstellen zum Weiterlernen.

Verzeichnis der Befehlsbeispiele

Beispiel 1.1: Typischer Ablauf einer Passwort-Prüfung (Konzept) … Seite 7
Beispiel 2.1: Derselbe Klartext ergibt immer denselben Hash (ohne Salt) … Seite 9
Beispiel 2.2: Derselbe Klartext mit unterschiedlichem Salt … Seite 9
Beispiel 2.3: Das Grundprinzip in Pseudocode … Seite 10
Beispiel 3.1: John the Ripper installieren … Seite 11
Beispiel 3.2: Testhashes aus frei erfundenen Passwörtern erzeugen … Seite 11
Beispiel 3.3: Eine kleine, eigene Wortliste anlegen … Seite 12
Beispiel 4.1: Ohne Angabe eines Modus – Johns Standardreihenfolge … Seite 13
Beispiel 4.2: Single-Crack-Modus gezielt einsetzen … Seite 13
Beispiel 4.3: Wortlisten-Modus mit eigener Testliste … Seite 13
Beispiel 4.4: Ergebnisse ansehen … Seite 14
Beispiel 4.5: Incremental-Modus, zeitlich begrenzt … Seite 14
Beispiel 5.1: Das Format explizit angeben, statt zu raten … Seite 15
Beispiel 5.2: /etc/shadow und /etc/passwd zu einem John-Format kombinieren … Seite 15
Beispiel 5.3: Hash aus einer passwortgeschützten ZIP-Datei extrahieren … Seite 16
Beispiel 5.4: Weitere gängige *2john-Werkzeuge … Seite 16
Beispiel 6.1: Wortliste mit Standardregeln kombinieren … Seite 17
Beispiel 6.2: Was Regeln aus einem einzigen Eintrag machen (Konzept) … Seite 17
Beispiel 6.3: Einen bestimmten Regelsatz auswählen … Seite 17
Beispiel 6.4: Eine eigene Regel in john.conf ergänzen (Auszug) … Seite 18
Beispiel 6.5: Die eigene Regel verwenden … Seite 18
Beispiel 7.1: Incremental-Modus mit begrenztem Zeichensatz und Länge … Seite 19
Beispiel 7.2: Verfügbare Incremental-Modi auflisten … Seite 19
Beispiel 7.3: Maskenbasierter Angriff … Seite 19
Beispiel 7.4: Maske mit festem Präfix kombinieren … Seite 20
Beispiel 8.1: Geschwindigkeit verschiedener Hash-Verfahren benchmarken … Seite 21
Beispiel 8.2: Mehrere CPU-Kerne nutzen … Seite 21
Beispiel 8.3: Einen benannten Lauf starten … Seite 21
Beispiel 8.4: Einen unterbrochenen Lauf fortsetzen … Seite 21
Beispiel 8.5: Mit der Potfile arbeiten … Seite 22
Beispiel 9.1: Wortlisten-Angriff in beiden Werkzeugen (Konzept, Syntax vereinfacht) … Seite 23
Beispiel 9.2: Maskenbasierter Angriff in beiden Werkzeugen (Konzept) … Seite 23
Checkliste 10.1: Vor jeder Übung … Seite 25
Beispiel 11.1: Die wichtigsten John-the-Ripper-Befehle auf einen Blick … Seite 29

Index

--format – 25, 26
--mask – 26
--rules – 26
--show – 14, 25
bcrypt – 7, 21, 26
Hashcat – 23, 27
Incremental-Modus – 13, 14, 19, 26, 27, 29
John the Ripper – 5, 7, 10, 11, 12, 23, 25, 27
Jumbo-Version – 16, 26, 29
Maske – 14, 19, 20, 23, 26, 29
Potfile – 14, 22, 26, 29
Rainbow Table – 9, 29
Regel – 5, 10, 13, 17, 18, 26, 29
Salting – 5, 9, 29
Session – 21, 27
Single Crack – 13, 25, 29
unshadow – 15, 26, 29
Wortliste – 5, 7, 10, 12, 13, 17, 18, 23
zip2john – 27

JOHN THE RIPPER DAS PRAXISHANDBUCH

Wie widerstandsfähig ist ein Passwort wirklich?

John the Ripper prüft offline beschaffte Passwort-Hashes gegen Wortlisten, Regeln und systematische Kombinationen. Dieses Handbuch erklärt Hashing und Salting an der Quelle, zeigt die drei grundlegenden Angriffsmodi, Regeln und Masken, und macht am eigenen Benchmark sichtbar, warum moderne Verfahren wie bcrypt einen so großen Unterschied machen. Durchgehend mit selbst erzeugten Testhashes, nie mit echten Zugangsdaten.

DAS ERWARTET DICH

- Hashing und Salting: warum Rainbow Tables an individuellem Salt scheitern
- Drei Angriffsmodi: Single Crack, Wortliste und Incremental im Vergleich
- Regeln und Masken: aus einer kleinen Wortliste gezielt mehr herausholen
- Hash-Extraktion mit unshadow und den *2john-Werkzeugen
- Performance-Vergleich, Sessions, Potfile-Verwaltung und Hashcat im Vergleich

Für wen?

Leser:innen der anderen Bände dieser Reihe, die die Authentifizierungsseite der IT-Sicherheit vertiefen möchten, sowie Studierende und Auszubildende. Ausschließlich für selbst erzeugte Testhashes und autorisierte Audits.

32 Seiten · Stand Oktober 2026 · Ergänzung zum Ethical-Hacking-, hping3-, Nmap- und Wireshark-Handbuch

Hinweis: Keine Rechts- oder Anlageberatung. Marken- und Produktnamen gehören ihren jeweiligen Inhabern.